Cave-gen-automaton

Project Proposal: One Rule Engine That Runs Game of Life and Builds Cave Levels
Name: Camellia Tran
Email: cametran@pdx.edu
Repository: https://github.com/Cameli19/cave-gen-automaton
Topic Area: Cellular Automata and Game Level Generation
Theme: Emergent Behavior

Project Description

- I want to build a Rust library and a small command-line program that creates patterns and cave maps using a grid of cells. Each cell is either alive (filled) or dead (empty).
- The program follows a set of simple rules to decide what happens to each cell. It looks at the eight cells around it and decides whether it should stay alive, become alive, or become empty. All cells update at the same time, and this process repeats over and over.
- I can change the rules to create different results. I will use a short format called B/S notation to describe them. For example, B3/S23 means:
    - **B3 (Birth):** An empty cell becomes alive if exactly 3 of its neighbors are alive.
    - **S23 (Survival):** A living cell stays alive if it has 2 or 3 living neighbors. Otherwise, it becomes empty.
- I will include two main settings:
    - **Game of Life (B3/S23):** Starts with random living and empty cells. Over time, interesting patterns appear. Some shapes stay in place, some blink on and off, and others move across the grid.
    - **Cave Rule (B5678/S5678):** Starts with random dots that gradually turn into cave-like maps. Empty spaces become connected rooms and tunnels, while living cells become walls or rocks.
- The interesting part is that I will use the same program for both. I only need to change the rules to get completely different results. This could be useful for game developers who want to create new cave levels automatically instead of designing every level by hand.

Relevance

The Simple Rules

- Each cell only looks at its eight neighbors to decide what happens next. It does not know what the rest of the

- grid looks like or what the final result should be.
- All cells update at the same time, based on the previous grid. For cave maps, spaces outside the grid count
- as rocks so the caves have closed boundaries. For Game of Life, spaces outside the grid count as empty.

What I Hope to See

- For Game of Life, I expect to see shapes that stay still, patterns that blink on and off, and shapes that move across the grid.
- For the cave rule, I expect random dots to gradually form smoother cave maps after about 4 or 5 rounds of updates.
- No cell knows what the final result will look like. The patterns and cave maps appear because many cells follow the same simple rules. Even a small change to the rules or the starting grid can create a very different result.

How I Will Show It

- The program will display the grid as a text map and save it as a PNG image.
- For Game of Life, it will show how many living cells remain after each round.
- For cave maps, it will report:
    - How much of the map is open space.
    - How many separate rooms there are.
    - The shortest path from the starting point to the exit.
    - How many treasures are placed in the cave.

My Work

Game of Life, B/S notation, and using cellular automata to create caves are already well-known ideas. However, I plan to write the code myself based on the rules and descriptions, rather than following an existing tutorial.
These are the main parts I will build:

- **A rule engine:** A program that reads rules like B3/S23 and uses them to update the grid. I will include Game of Life and the cave rule
- **A treasure cave level builder:** After creating a cave, the program will find all the separate rooms and keep the largest connected room. It will place the starting point on one side of the cave. Then it will search for the farthest reachable spot to use as the exit. It will also find the shortest path between the start and exit. Lastly, it will place treasures in dead ends, which are small paths that do not lead anywhere else.
- **A seed option:** A seed is a starting number used to create the random grid. Using the same seed will create the same starting grid, making it easier to test and compare results.

Plan

The project will be a Rust library with a small command-line program that lets people run it.
I plan to organize the code into these parts:

- **rule:** Reads and understands the rules.
- **grid:** Creates the cells and updates them.
- **rooms:** Finds separate rooms and connected spaces.
- **level:** Places the start, exit, path, and treasures.
- **render:** Displays the grid as text and saves it as an image.
- **main.rs:** Handles the commands and options the user enters.

I plan to use three small Rust libraries (crates): rand for creating random grids, clap for handling command-line options, and image for saving PNG files.
I expect the project to be around 600 to 1,000 lines of code.

Testing and Code Quality

I will write tests to make sure the program works correctly. Some examples include:

- The rule reader accepts correctly written rules and rejects incorrect ones.
- A blinker (a pattern of three cells in a row) switches between horizontal and vertical every round.
- A glider moves diagonally across the grid over time.
- Using the same seed creates the same starting grid.

I will also use cargo fmt to keep the code organized and cargo clippy to check for common coding mistakes. I will add Rust documentation to the parts of the library that other people can use. I will also use Git commits to show how the project develops over time.

Concerns

- **Time:** I am still learning Rust and have limited time each week. I will build the project in small steps, starting with the rule engine and Game of Life so I have something working early. If I run out of time, I will focus on finishing the engine rather than the additional cave features.
- **Originality:** Game of Life and cave generation are well-known ideas. My goal is to write my own rule engine and combine it with a cave level builder that adds exits, paths, and treasures.
- **Learning Rust:** I expect to encounter some compiler errors as I learn Rust, but I will work through them step by step.

Sources

Game of Life and B/S notation are established concepts. The cave generation idea is described in RogueBasin's article, "Cellular Automata Method for Generating Random Cave-Like Levels."
I will also use standard algorithms such as flood fill (finding all connected open spaces) and breadth-first search (finding the shortest path between two points).
For learning and reference, I will use *The Rust Programming Language* book, and the documentation for the libraries I use. I will write the project code myself.